

Chapter 2

BASIC ELEMENTS IN C++

Chapter 2

- **Program structures**
- **Data types and operators**
- **Variables and declaration statements**

1. Program Structure

- A C++ program is a collection of functions.
- *A function* is a program segment that transforms the data it receives into a finished result.

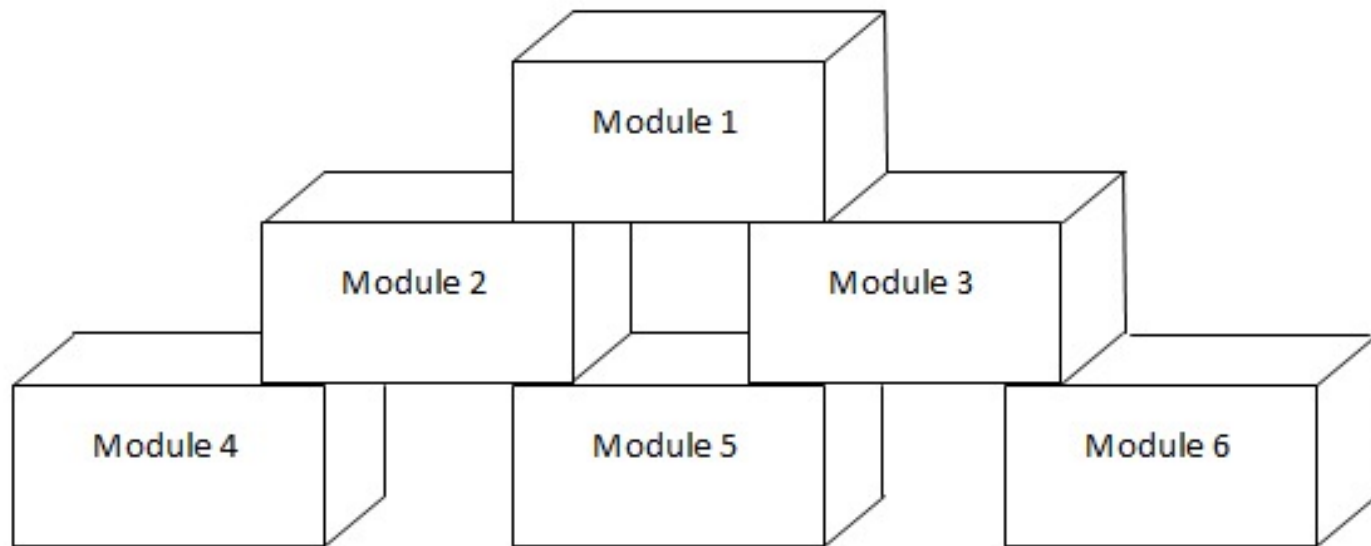


Fig. 1. A well-designed program is built using modules

Function

- Each function must have a name.
- Names or *identifiers* in C++ can be made up of any combination of letters, digits, or underscores selected according to the following rules:
 - Identifiers must begin with an uppercase or lowercase ASCII letter or an underscore (_).
 - You can use digits in an identifier, but not as the first character. You are not allowed to use special characters such as \$, &, * or %.
 - Reserved words cannot be used for variable names.

- **Example:**

Pascal: ThisIsAFunction

Camel: thisIsAFunction

DegToRad

intersect

addNums

FindMax1

_density

slope

The *main()* function

- The *main()* function is a special function that runs automatically when a program first executes.
- All C++ programs must include one *main()* function. All other functions in a C++ program are executed from the *main()*.
- The first line of the function, in this case *int main()* is called a *function header line*.
- The function header line contains three pieces of information:
 1. What type of data, if any, is returned from the function.
 2. The name of the function
 3. What type of data, if any, is sent into the function.

```
int main()
```

```
int getNumber(void)
```

```
int totalScore(int score1, int score2)
```

The *main()* function (cont.)

```
int main()
{

    program statements in here
    return 0;
}
```

The line

return 0;

is included at the end of every *main* function. C++ keyword *return* is one of several means we will use to *exit a function*. When the *return* statement is used at the end of *main()*, the value *0* indicates that the program has **terminates successfully**.

The `cout` Object

- The *cout* object is an output object that sends data given to it to the standard output display device.
- To send a message to the `cout` object, you use the following pattern:

```
cout << "Hello " << "C++";
```



```
cout << "text";
```
- The *insertion operator*, `<<`, is used for sending text to an output device.
- The text portion of *cout* statement is called a *text string*.

A simple program

Example 2.1.1

```
#include <iostream>
```



header file

```
using namespace std;
```

```
int main()
```

```
{
```

```
    cout << "Hello world!";
```

```
    return 0;
```

```
}
```

- A *header file* is a file with an extension of *.h* that is included as part of a program. It notifies the compiler that a program uses run-time libraries.
- All statements in C++ must end with a semicolon.
- The *iostream* classes are used for giving C++ programs input and output capabilities.
- The header file for the *iostream* class is *iostream.h*.

The iostream classes

- The ***#include*** statement is one of the several *preprocessor directives* that are used with C++.

Example: To include the *iostream.h* file you use the following *preprocessor directives*:

`#include <iostream>`

The statement `using namespace std;`
tell the compiler where to find the header files.

A *namespace* is a specific named section of code within a folder that is accessed by the compiler when it is looking for prewritten classes or function.

Preprocessor directives

- The *preprocessor* is a program that runs before the compiler.
- When the preprocessor encounters an *#include* statement, it places the entire contents of the designated file into the current file.
- Preprocessor directives and *include* statements allow the current file to use any of the *classes*, *functions*, *variables*, and *other code* contained within the included file.

i/o manipulator

- **An *i/o manipulator* is a special function that can be used with an i/o statement.**
- **The *endl* i/o manipulator is part of *iostream* classes and represents a new line character.**
- **Example:**

```
cout << "Program type: console application" << endl;  
cout << "Create with: Visual C++ " << endl;  
cout << "Programmer: Don Gesselin" << endl;
```

Comments

- **Comments** are lines that you place in your code to contain various type of remarks.
- **C++ *line comments*** are created by adding two slashes (`//`) before the text you want to use as a comment.

Ex:

`// this program calculates a square root`

- **Block comments** span multiple lines. Such comments begin with `/*` and end with the symbols `*/`.

Ex:

`/* this program solves a quadratic equation:`

`ax2 + bx +c = 0`

`*/`

2. DATA TYPES AND OPERATORS

Data Types

- A *data type* is the specific category of information that a variable contains.
- There are three basic data types used in C++: integers, floating point numbers and characters.

Integer Data Type

- An integer is a positive or negative number with no decimal places.
- Examples:
- 259 -13 0 200

Floating Point Numbers

- A *floating point* number contains decimal places or is written using exponential notations.

-6.16 -4.4 2.7541 10.5
90.1e3 0.89E8 -1.01e-1

Note: 90.1e3 means 90.1×10^3

The Character Data Type

- To store text, you use the character data type. To store one character in a variable, you use the *char* keyword and place the character in single quotation marks.
- Example:
`char cLetter = 'A';`
- Escape Sequence

The combination of a *backlash* (\) and a special character is called an escape sequence.

- Example:
 `\n` move to the next line
 `\t` move to the next tab

Arithmetic Operators

- Arithmetic operators are used to perform mathematical calculations, such as addition, subtraction, multiplication, and division.

Operator	Description

+	Add two operands
-	Subtracts one operand from another operand
*	Multiplies one operand by another operand
/	Divides one operand by another operand
%	Divides two operands and returns the remainder

- A simple arithmetic expression consists of an arithmetic operator connecting two operands in the form:
operand1 operator operand2

Examples:

3 + 7

18 – 3

12.62 + 9.8

12.6/2.0

Example 2.2.1

```
#include <iostream>
using namespace std;
int main()
{
    cout << "15.0 plus 2.0 equals " << (15.0 + 2.0) << '\n'
        << "15.0 minus 2.0 equals " << (15.0 - 2.0) << '\n'
        << "15.0 times 2.0 equals " << (15.0 * 2.0) << '\n'
        << "15.0 divided by 2.0 equals " << (15.0 / 2.0) << '\n';
    return 0;
}
```

The output of the program:

15.0 plus 2.0 equals 17

15.0 minus 2.0 equals 13

15.0 times 2.0 equals 30

15.0 divided by 2.0 equals 7.5

Integer Division and % operator

- The division of two integers yields integer result. Thus the value of $15/2$ is 7.
- Modulus % operator produces the remainder of an integer division.
- Example:
 - $9\%4$ is 1
 - $17\%3$ is 2
 - $14\%2$ is 0

Operator Precedence and Associativity

- Expressions containing multiple operators are evaluated by the priority, or *precedence*, of the operators.

Operator	Associativity

unary -	Right to left
* / %	Left to right
+ -	Left to right

- Example:

8 + 5*7%2*4
↓ ↓ ↓ ↓
4 1 2 3

3. VARIABLES

- One of the most important aspects of programming is storing and manipulating the values stored in *variables*.
- Variable names are also selected according to the rules of identifiers:
 - Identifiers must begin with an uppercase or lowercase ASCII letter or an underscore (_).
 - You can use digits in an identifier, but not as the first character. You are not allowed to use special characters such as \$, &, * or %.
 - Reserved words cannot be used for variable names.

Identifiers

- **Example: Some valid identifiers**

my_variable (python style)

MyVariable (pascal style)

myVariable (camel style 🐪, suggested style for C++)

Temperature

x1

x2

_my_variable

- **Some invalid identifiers are as follows:**

%x1 %my_var @x2

Declaration Statements

- In C++ you can declare the data types of variables using the syntax:

type name;

The *type* portion refers to the data type of the variable.

- The data type determines the type of information that can be stored in the variable.
- Example:
 int count;
 float sum;

Rules of variable declaration

■ Rules:

1. A variable must be declared before it can be used.
2. Declaration statements can also be used to store an initial value into declared variables.

Example:

```
int num = 15;  
float grade1 = 87.0;
```

- **Note: Declaration statement gives information to the compiler**
 - rather than a step in the algorithm.

Example 2.2.1

```
#include <iostream>
using namespace std;
int main()
{
    float grade1 = 85.5;
    float grade2 = 97.0;
    float total, average;
    total = grade1 + grade2;
    average = total/2.0; // divide the total by 2.0
    cout << "The average grade is " << average << endl;
    return 0;
}
```

The output of the above program:

The average grade is 91.25

Let notice the two assignment statements in the above program:

total = grade1 + grade2; *average = total/2.0;*

- Each of these statements is called an *assignment statement* because it tells the computer to assign (store) a value into a variable.